

Chapter 11. Operation Contracts

“When ideas fail, words come in very handy.”

Objectives

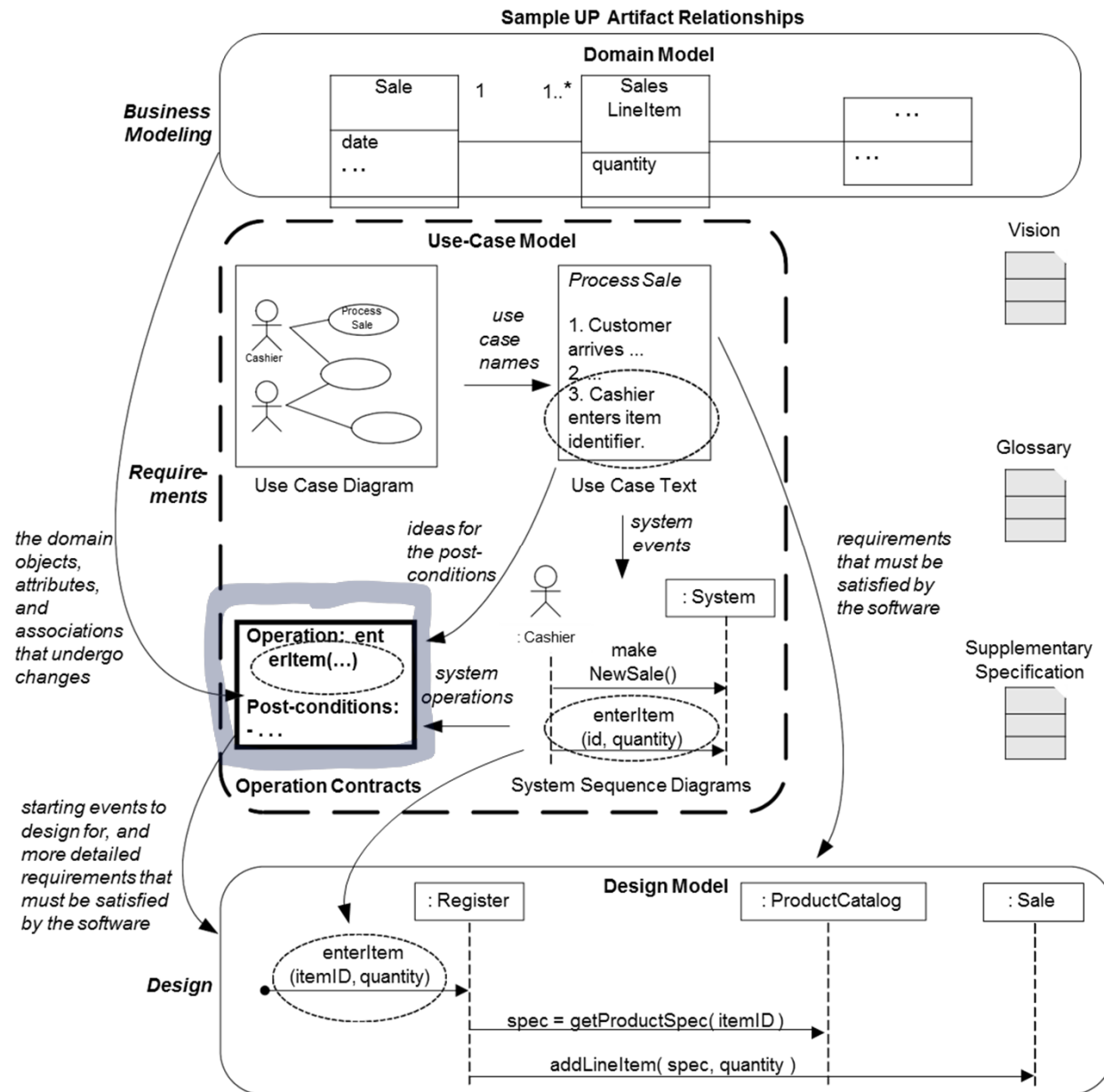
- Define system operations.
- Create contracts for system operations.

Operation Contracts

→ 사전/사후 형식으로 system operation
실행 결과로 도메인 모델의 객체가

어떻게 변화하는지 기술하는 문서

- Operation contracts
 - use a pre and post-condition form to describe detailed changes to objects in a domain model as the result of a system operation.
- Usually, they are used in a Design Model for object methods
- But, they can also be used in a Domain Model as contracts of high-level system operations.



Operation Contract Example

- An **operation contract** for the **enterItem** system operation.

여기

Operation:	enterItem(itemID: ItemID, quantity: integer)
Cross References:	Use Cases: Process Sale
Preconditions:	There is a sale underway.
Postconditions:	- A SalesLineItem instance sli was created (instance creation). - sli was associated with the current Sale (association formed). - sli.quantity became quantity (attribute modification). - sli was associated with a ProductDescription, based on itemID match (association formed).

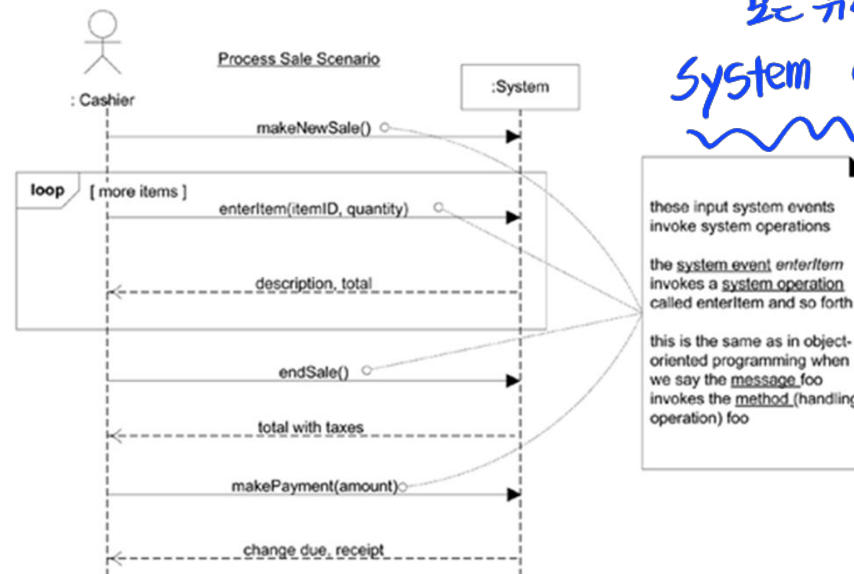
포맷

Operation:	Name of operation, and parameters
Cross References:	Use cases this operation can occur within
Preconditions:	Noteworthy assumptions about the state of the system or objects in the Domain Model before execution of the operation. These are non-trivial assumptions the reader should be told.
Postconditions:	The most important section. The <u>state of objects in the Domain Model</u> after completion of the operation.



What is a System Operation?

- System operations
 - Operations that the system as a black box component offers in its public interface → 시스템을 블랙박스 컴포넌트로 볼 때, public interface로 제공하는 연산.
 - Can be identified while sketching SSDs
 - Show system events - events or I/O messages relative to the system.
 - Input system events imply that the system has system operations to handle the events.
- **System Interfaces**: the entire set of system operations across all use cases
 - Viewing the system as a single component or class.



모든 유스케이스에 걸친
system operations의 집합

these input system events
invoke system operations
the system event *enterItem*
invokes a system operation
called *enterItem* and so forth
this is the same as in object-
oriented programming when
we say the message *foo*
invokes the method (handling
operation) *foo*

Postconditions

- The **postconditions** describe changes in the state of objects in the domain model. The changes include:
 - instances created or deleted
 - associations formed or broken
 - attributes changed
- main purpose: sometimes the effect of a system operation is not clear to the developer, and more detail and precision is useful and required.
 - Note: postconditions are only necessary when the outcome of a system operation is not clear from the use case description.

↳ usecase 설명만으로 연산 결과가
명확하지 않을 때 사용.
명확하면 굳이 postcondition 쓸 필요 X

How to Write a Postcondition?

→ 과거형으로 기술.

- Express postconditions **in the past tense**.
 - To emphasize that they are observations about state changes that arose from an operation, not an action to happen.
 - That's why they are called **postconditions**.
 - Example:
 - (better) A *SalesLineItem* **was** created.
 - (worse) ~~Create~~ a *SalesLineItem*. → 현재형 X
 - A *SalesLineItem* ~~is~~ created. → 현재형 X

Example: *EnterItem* Postconditions

- **Instance Creation and Deletion**

- After the *itemID* and *quantity* of an item have been entered, what new object should have been created?
- A *SalesLineItem*.
- Thus:
 - A *SalesLineItem* instance *sli* was created (instance creation).

- **Attribute Modification**

- After the *itemID* and *quantity* of an item have been entered, what attributes of new or existing objects should have been modified?
- The *quantity* of the *SalesLineItem* should have become equal to the *quantity* parameter.
- Thus:
 - *sli.quantity* became *quantity* (attribute modification).

Example: *EnterItem* Postconditions

- **Associations Formed and Broken**
 - After the *itemID* and *quantity* of an item have been entered, what associations between new or existing objects should have been formed or broken?
 - The new *SalesLineItem* should have been related to its *Sale*, and related to its *ProductDescription*.
 - Thus:
 - *sli* was associated with the current *Sale* (association formed).
 - *sli* was associated with a *ProductDescription*, based on *itemID* match (association formed).

Chapter 12. Requirements to Design Iteratively

“Hardware, n.: The parts of a computer systems that can be kicked.”

Objectives

- Quickly motivate the transition to design activities.
- Contrast the importance of object design skill versus UML notation knowledge.

Iteratively Do the Right Thing and Do the Thing Right

옳은 것을 하기 (목표·규칙·제약 이해)

- **Do the right thing:**

- The requirements and OOA have focused on determining to do the right thing. (무엇을 것인가?; what 관점)
- Understanding some outstanding goals, related rules and constraints.

올바르게 하기 (요구사항을 만족하는 솔루션을 잘 설계하기)

- **Do the thing right:**

- Design work will stress do the thing right.
- Skillful designing a solution to satisfy the requirements for its iteration. (어떻게 할 것인가; how 관점)

- In iterative development, a transition from requirements/OOA to design/implementation occur in each iteration.

↳ 매 iteration마다
요구사항/OOA는 전환이 일어난다.

Provoking Early Change 초기에 변화를 이끌어내라,

- Discover and change some requirements during the design and implementation work
 - Especially in the early iterations
- Embrace Change!
 - Early programming, tests, and demos help provoke the inevitable changes early on.
> (주요한 변화 요구사항은 elaboration의 앞부분 iter에 배치.)
 - Heart of why iterative development works
- [By the end of elaboration, 80% of the requirements are reliably defined]